

1. [2p] Dado el siguiente código Java,

- Dibujar un diagrama de clases UML que muestre todas las clases e interfaces involucradas (salvo las de la biblioteca estándar) y sus relaciones.
- Dibujar un diagrama de secuencia UML que muestre la interacción entre los objetos durante la ejecución de main.
- ¿Hay comportamiento polimórfico en el código? Indicar dónde y justificar.

Procesador.java

```
1 public interface Procesador {
2     String procesar(String texto);
3 }
```

EnMayusculas.java

```
1 public class EnMayusculas implements Procesador {
2     @Override
3     public String procesar(String texto) {
4         return texto.toUpperCase();
5     }
6 }
```

Invertir.java

```
1 public class Invertir implements Procesador {
2     @Override
3     public String procesar(String texto) {
4         return new StringBuilder(texto)
5             .reverse().toString();
6     }
7 }
```

Pipeline.java

```
1 import java.util.List;
2 import java.util.ArrayList;
3
4 public class Pipeline {
5     private List<Procesador> pasos = new ArrayList<>();
6
7     public void agregar(Procesador p) {
8         pasos.add(p);
9     }
10
11     public String ejecutar(String entrada) {
12         String resultado = entrada;
13         for (Procesador p : pasos) {
14             resultado = p.procesar(resultado);
15         }
16         return resultado;
17     }
18 }
```

Main.java

```
1 public class Main {
2     public static void main(String[] args) {
3         Pipeline pipeline = new Pipeline();
4         pipeline.agregar(new EnMayusculas());
5         pipeline.agregar(new Invertir());
6         System.out.println(
7             pipeline.ejecutar("hola")
8         );
9     }
10 }
```

Criterios de corrección:

• Diagrama de clases:

- Procesador es una interfaz; EnMayusculas e Invertir la implementan (flechas de realización).
- Pipeline tiene una composición/agregación con List<Procesador>. Es aceptable dibujar la asociación directamente hacia Procesador.
- Main tiene dependencias hacia Pipeline, EnMayusculas e Invertir (las instancias en main).
- No es necesario incluir List / ArrayList.
- No es necesario incluir los atributos y métodos.

• Diagrama de secuencia:

- Los participantes son: :Main, pipeline:Pipeline, em:EnMayusculas, inv:Invertir.
- Se debe ver la creación del pipeline, el agregar de cada procesador, y la llamada a ejecutar("hola").
- Dentro de ejecutar, se debe ver el loop (o las 2 iteraciones) con:
 - em.procesar("hola") → retorna "HOLA"
 - inv.procesar("HOLA") → retorna "ALOH"
- ejecutar retorna "ALOH", que luego se imprime por pantalla.

• Polimorfismo:

- Sí: en el for (Procesador p : pasos), la llamada p.procesar(resultado) es polimórfica.
- El tipo estático de p es Procesador (la interfaz), pero el tipo dinámico varía en cada iteración (EnMayusculas o Invertir).

2. [1p] Considerar el siguiente código. Identificar un principio de diseño que se viola, indicar cuál es y justificar. Proponer una solución para evitar la violación del principio (no es necesario escribir el código).

```
1 public class ClienteEmailSMTP {
2     public void enviar(String dest, String cuerpo) {
3         // conecta al servidor SMTP y envía el email
4     }
5 }
6
7 public class ArchivoConfig {
8     public String destinatarioAlertaEmail() {
9         // lee la configuración del disco
10    }
11 }
```

```
12 public class ServicioAlertas {
13     private ClienteEmailSMTP email = new
        ClienteEmailSMTP();
14     private ArchivoConfig config = new ArchivoConfig();
15
16     public void alertar(String mensaje) {
17         String dest = config.destinatarioAlertaEmail();
18         if (dest != null) {
19             email.enviar(dest, mensaje);
20         }
21     }
22 }
```

Criterios de corrección:

- El principio violado es el **Principio de Inversión de Dependencias (DIP)**: ServicioAlertas (módulo de alto nivel) instancia directamente EmailSender y ArchivoConfig (módulos de bajo nivel concretos) en lugar de depender de abstracciones.
- Se puede considerar otro principio como correcto si está bien justificado.
- Solución: extraer interfaces (e.g. Notificador y Config) e inyectarlas en ServicioAlertas por constructor o setter.
- Verificar que usen los términos correctos en la justificación y la solución propuesta.

3. [2p] Dado el siguiente código Prolog,

- a) Explicar cómo se resuelve la consulta `?- contiene(a, [a, b, c]).`
- b) ¿El resultado es el esperado?
- c) ¿Qué cambiaría si se invierte el orden de las líneas 1 y 2?

```
1 | contiene(X, [_|T]) :- contiene(X, T).  
2 | contiene(X, [X|_]).
```

Criterios de corrección:

- **a)** Prolog intenta la cláusula 1 primero (la recursiva): descarta el primer elemento e intenta `contiene(a, [b, c])`, luego `contiene(a, [c])`, luego `contiene(a, [])`. La lista vacía no unifica con ninguna cláusula: falla. Prolog hace *backtracking*: en `contiene(a, [c])` intenta la cláusula 2 (`c = a`, falla), en `contiene(a, [b, c])` intenta la cláusula 2 (`b = a`, falla), y finalmente en `contiene(a, [a, b, c])` intenta la cláusula 2 (`a = a`, éxito). Retorna `true`.
- **b)** Sí, el resultado es el esperado: `a` pertenece a la lista y la consulta retorna `true`.
- **c)** Para esta consulta concreta el resultado es el mismo, pero se encuentra de forma más eficiente (la cláusula 1 unifica directamente, sin recursar). La diferencia observable es en consultas con múltiples soluciones: con el orden original `?- contiene(X, [a, b, c]).` devuelve `c, b, a` (orden inverso); con el orden invertido devuelve `a, b, c` (orden natural), porque la cláusula base se intenta primero en cada nivel.

4. [2p] Dado el siguiente código Clojure,

- a) ¿Qué imprime el programa al invocar (main)?
- b) ¿Cuáles funciones (con nombre o anónimas) son puras/impuras? Justificar.
- c) ¿Cuáles son de orden superior? Justificar.
- d) ¿Hay alguna clausura? Explicar qué significa esto y dónde ocurre en el código (si es que hay alguna).

```
1 (defn multiplicador [factor]
2   (fn [x] (* x factor)))
3
4 (defn main []
5   (let [doble (multiplicador 2)
6         triple (multiplicador 3)]
7     (println (map doble [1 2 3]))
8     (println (map triple [1 2 3]))))
```

Criterios de corrección:

- **a)** Imprime:
(2 4 6)
(3 6 9)
- **b)** Puras: multiplicador (dado el mismo factor siempre retorna la misma función, sin efectos secundarios) y la función anónima (fn [x] (* x factor)) (solo realiza una multiplicación). Impura: main (invoca println, que produce salida por pantalla).
- **c)** De orden superior: multiplicador (retorna una función). También map (recibe una función como argumento).
- **d)** Una clausura es una función que captura variables del entorno donde fue definida. La fn [x] captura factor del ámbito de multiplicador y lo recuerda incluso después de que multiplicador haya retornado. Por eso doble siempre multiplica por 2 y triple siempre multiplica por 3, de forma independiente.

5. [2p] Dada la siguiente expresión lambda,

- Identificar las variables libres y ligadas.
- Mostrar los pasos de reducción usando orden normal (β -redex más externa desde la izquierda) hasta llegar a la forma normal (si existe).
Aplicar reducciones α según sea necesario para evitar conflictos de nombres.

$(\lambda x. \lambda y. x \ y) \ (\lambda z. z \ y)$

Criterios de corrección:

- Variables ligadas: x (ligada por λx), y (ligada por λy), z (ligada por λz). Variable libre: y (la que aparece en el cuerpo de $\lambda z. z \ y$).
- Reducción:
 - α : renombrar y a w en $\lambda y. x \ y$:
 $(\lambda x. \lambda w. x \ w) \ (\lambda z. z \ y)$
 - β :
 $\lambda w. (\lambda z. z \ y) \ w$
 - β :
 $\lambda w. w \ y$

6. [1p] Dado el siguiente código Java,

- Explicar cómo funciona el programa. ¿Qué ventaja tiene esta implementación sobre una suma lineal?
- ¿La salida es determinística? Justificar.

Sumador.java

```
1 import java.util.concurrent.RecursiveTask;
2
3 public class Sumador extends RecursiveTask<Integer> {
4     private static final int UMBRAL = 2;
5     private final int[] arr;
6     private final int inicio, fin;
7
8     public Sumador(int[] arr, int inicio, int fin) {
9         this.arr = arr;
10        this.inicio = inicio;
11        this.fin = fin;
12    }
13
14    @Override
15    protected Integer compute() {
16        if (fin - inicio <= UMBRAL) {
17            int suma = 0;
18            for (int i = inicio; i < fin; i++)
19                suma += arr[i];
20            return suma;
21        }
22        int medio = (inicio + fin) / 2;
23        Sumador izq = new Sumador(arr, inicio, medio);
24        Sumador der = new Sumador(arr, medio, fin);
25        izq.fork();
26        return der.compute() + izq.join();
27    }
28 }
```

Main.java

```
35 import java.util.concurrent.ForkJoinPool;
36
37 public class Main {
38     public static void main(String[] args) {
39         int[] arr = {1, 2, 3, 4, 5, 6, 7, 8};
40         ForkJoinPool pool = new ForkJoinPool();
41         System.out.println(
42             pool.invoke(
43                 new Sumador(arr, 0, arr.length)
44             )
45         );
46     }
47 }
```

Crterios de corrección:

- a) El programa calcula la suma de un arreglo de forma recursiva: si el subarreglo tiene tamaño menor o igual al umbral (UMBRALE = 2), lo suma secuencialmente. Si no, divide el arreglo en dos mitades: la izquierda se delega al pool con `fork()` (potencialmente en otro hilo) y la derecha se computa en el hilo actual con `der.compute()`; luego `izq.join()` espera y obtiene el resultado del hilo delegado. La ventaja sobre una suma lineal es que las mitades pueden ejecutarse en paralelo en distintos núcleos, reduciendo el tiempo total para arreglos grandes.
- b) Sí, la salida es determinística: siempre imprime 36 (= 1 + 2 + ... + 8). Aunque el orden de ejecución de los hilos es no determinístico, la adición es conmutativa y asociativa, por lo que el resultado final es siempre el mismo.